

บทที่ 4

เครื่องมือที่นำมาประยุกต์ใช้งาน

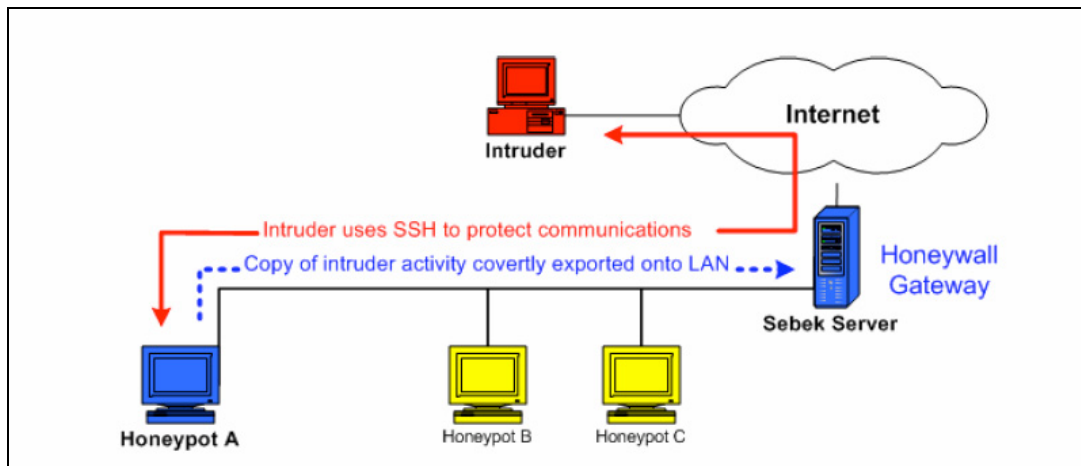
เนื่องจากโครงงาน ชุดโปรแกรมจำแนกและบันทึกพฤติกรรมผู้บุกรุก ประกอบขึ้นด้วยส่วนประกอบหลายๆส่วน มีหน้าที่การทำงานในหลายๆด้าน จึงจำเป็นที่จะต้องใช้เครื่องมือที่เหมาะสม และสามารถทำงานได้อย่างที่ผู้พัฒนาคาดหวัง โดยแต่ละเครื่องมือก็จะมีจุดเด่น และข้อจำกัดในการใช้งาน ซึ่งการนำมาใช้งานร่วมกัน ให้สามารถสื่อสารและเชื่อมโยงกันได้นั้น จำเป็นที่จะต้องนำเครื่องมือต่างๆมาปรับปรุงและประยุกต์ใช้ เพื่อให้ได้ประสิทธิภาพของระบบที่ดีที่สุดโดยในบทนี้จะให้รายละเอียด ของแต่ละเครื่องมือที่นำมาประยุกต์ใช้งานในระบบ

เครื่องมือต่างๆ ที่ได้นำมาประยุกต์ใช้งาน

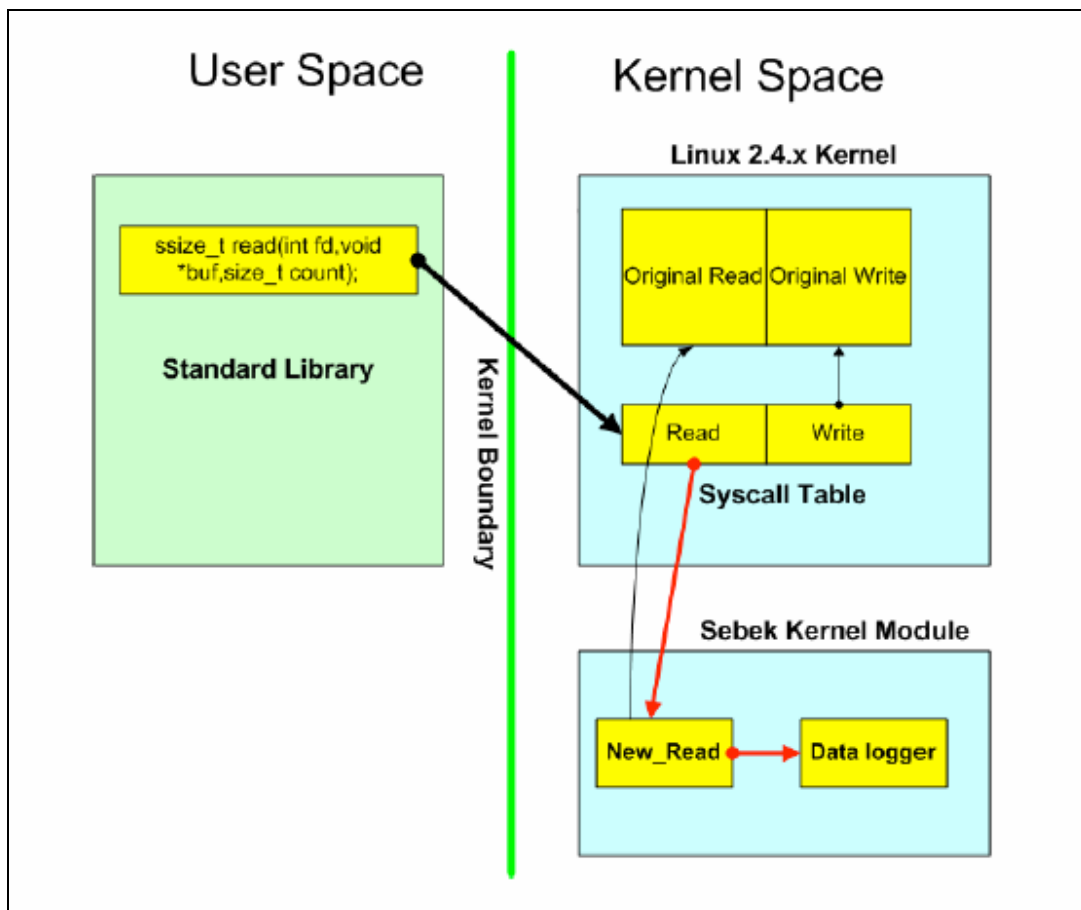
- Sebek
- Samhain
- Snort inline
- IPsec
- S2I
- Iptables

4.1 Sebek

เซเบคไคลเอนจะคอยซ่อนตัวอยู่ในกับดักเพื่อเฝ้าดูพฤติกรรมและการกระทำของผู้บุกรุก และคอยส่งข้อมูลนั้นไปยังเครื่องที่ใช้เก็บพฤติกรรมลงฐานข้อมูล โดยส่งผ่านช่องทางที่มีความปลอดภัย มีการเข้ารหัสข้อมูลในรูปแบบของ เซเบคเอง

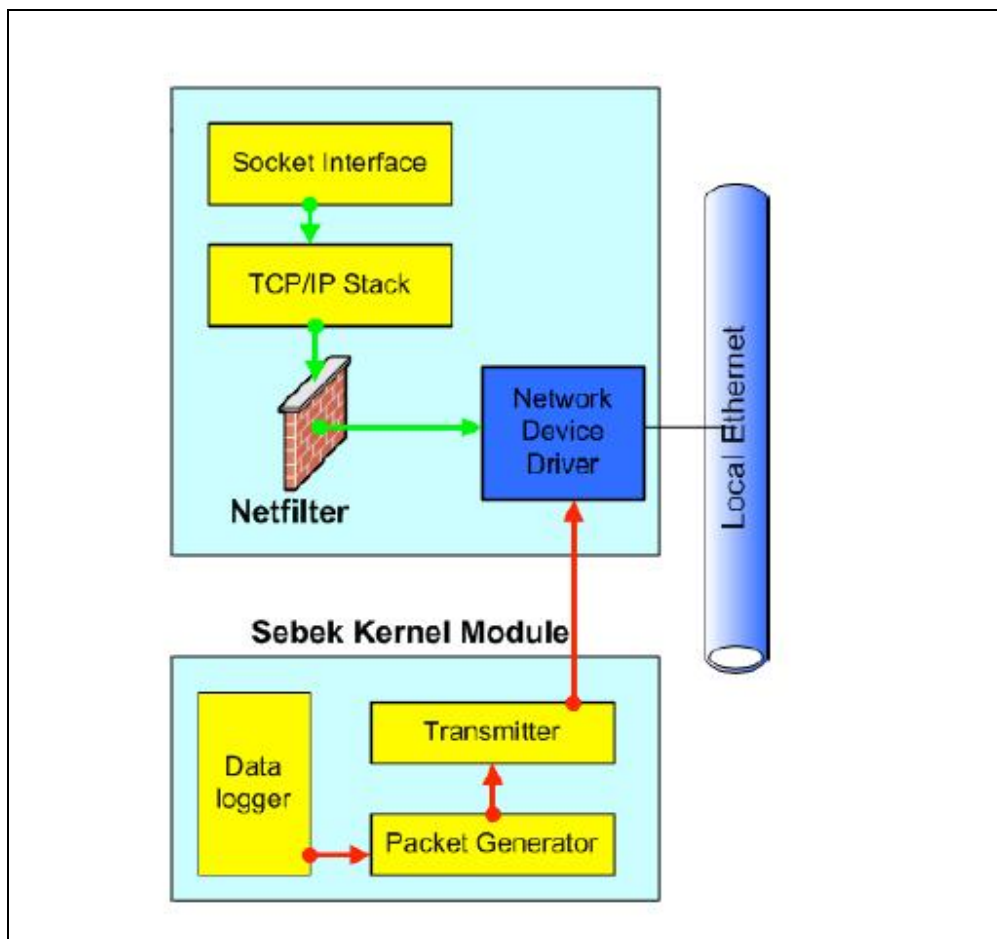


รูปที่ 4.1 แสดงการทำงานของ Sebek



รูปที่ 4.2 รูปภาพแสดงการcapture ข้อมูลที่ทำภายใน kernel

เซเบกเป็นData Capture Toolแบบ Kernel-based โดยในการนำมาใช้จะประยุกต์ใช้เป็นแบบ LKM (Loaded Kernel Module) มีความโดดเด่นในความสามารถในการดักจับข้อมูลได้ทุกการกระทำของIntruderที่กระทำผ่าน read() system call ไม่ว่าจะเป็นในส่วนของ keystroke, files transfer, burneye passwords หรือหากมีการใช้ IRC clientหรือ e-mail client ตัวเซเบกก็สามารถดักจับได้ ดังเช่นเมื่อIntruderมีการนำfileส่งเข้ามา ตัวเซเบกก็สามารถอ่านและบันทึกไฟล์นั้นไว้ได้ ทั้งยังสามารถใช้เป็นGlass-box เมื่อเปรียบเทียบกับการทำงานของblack-boxที่เรารู้จัก นั่นคือเราสามารถติดตามการทำงานของโปรแกรมที่intruderเอาจริงได้ด้วย แม้ว่าตัวintruder จะlog out ออกไปแล้ว



รูปที่ 4.3 รูปภาพแสดงการby-passไม่ผ่าน TCP stack

เซเบกมีรูปแบบวิธีการปิดซ่อนตัวเองได้อย่างมีประสิทธิภาพ และค่อนข้างยากที่จะตรวจสอบพบทั้งจากภายในและภายนอก โดยจะทำการปิดซ่อนโมดูลที่ใช้ และลบlinked list ที่จะแสดงการมี

ตัวตนออก โดยใช้ the cleaner ทำให้Intruder ไม่สามารถตรวจสอบพบว่าเซเบคกำลังทำงานอยู่ภายในเครื่อง ทุกpacketของเซเบค ตัวเซเบคจะมีfunctionในการสร้างและส่งเอง โดยไม่ใช้ TCP/IP stack ดังนั้นจึงไม่สามารถถูกมองเห็นและไม่สามารถblock traffic ของเซเบคได้ หลังจากสร้างpacketเสร็จ packetจะถูกส่งไปยังdevice driverโดยตรง ไม่ผ่าน raw socket socket interface ซึ่งพวกโปรแกรม sniffer ที่ใช้ libpcap เป็นฐานจะไม่สามารถเห็นได้เพราะมันไปดักจับข้อมูลที่ raw socket interface และเพื่อป้องกันการตรวจจับในเครือข่าย sebekจะไม่ใช้ARP แต่จะกำหนดMAC Addressของserver ไว้เลย เพื่อไม่ต้องทำการ ARP request เพื่อป้องกันการทำ ARP spoofing ซึ่งก็จะลดการดักจับข้อมูลในเครือข่ายได้ ทั้งนี้เซเบคมีรูปแบบการส่งข้อมูลด้วยโปรโตคอล UDP

4.2 Samhain

Samhain เป็นระบบที่ใช้ในการตรวจสอบความถูกต้องของข้อมูลและใช้ในการแจ้งเตือนการบุกรุกระบบซึ่งสามารถใช้บนเครื่องโฮสต์เครื่องเดียว หรือใช้ในระบบเครือข่ายขนาดใหญ่ก็ได้ ซึ่งในโครงการนี้ไม่ได้ใช้กับเครื่องโฮสต์เพียงเครื่องเดียว หากแต่จะใช้กับเครื่องหลายๆ เครื่องที่อยู่ในระบบเครือข่ายโดยจะมีเครื่องแม่ข่ายที่จะทำหน้าที่เป็นศูนย์กลางคอยรับข้อมูลจากเครื่องลูกข่ายทั้งหลายในระบบเครือข่าย ซึ่งจะได้อธิบายรายละเอียดของโครงสร้างและการทำงานในส่วนถัดไป

4.2.1 การใช้งานในรูปแบบระบบเครือข่าย (network)

Samhain เป็นระบบที่ถูกออกแบบมาให้ช่วยต่อการมอนิเตอร์โฮสต์หลายๆ เครื่องในเครือข่าย ซึ่งมันจะประกอบไปด้วยโปรเซสที่ทำงานในลักษณะที่เป็นเดมอนในโฮสต์แต่ละเครื่อง และจะมีเครื่องแม่ข่ายกลางที่จะคอยเก็บบันทึกหรือรายงานต่างๆ จากเดมอนเหล่านั้นผ่านทาง การเชื่อมต่อแบบ ทีซีพี/ไอพี ซึ่งในการส่งข้อมูลจากเครื่องลูกข่ายต่างๆ ไปให้เครื่องแม่ข่ายกลางนั้นก็จะต้องมีการยืนยันตนก่อนเพื่อป้องกันการปลอมแปลงข้อความที่ส่งไปให้เครื่องแม่ข่าย โดยในขั้นแรกจะต้องตกลงกันในเรื่องของโปรโตคอลที่จะใช้ในการยืนยันตนก่อน จากนั้นจะมีการแลกเปลี่ยน session key กัน การเชื่อมต่อที่มาจากเครื่องโฮสต์ที่ไม่ได้ทำการลงทะเบียนไว้จะถูกครอบกั้นทันที และในอีกกรณีหนึ่งที่จะถูกครอบกั้นคือการเชื่อมต่อจากเครื่องโฮสต์ที่ได้มีการลงทะเบียนไปแล้วแต่เครื่องลูกข่ายไม่สามารถทำการยืนยันตนได้สำเร็จ

เมื่อ Session key ได้ถูกสร้างขึ้นมาเครื่องลูกข่ายจะใช้ session key นั้นในการเซ็นลงไปในข้อความของตนเพื่อใช้ในการยืนยันตนและเมื่อข้อความถูกส่งไปถึงเครื่องแม่ข่าย เครื่องแม่ข่ายจะทำ

การตรวจสอบลายเซ็นของเครื่องลูกข่ายจาก เมื่อตรวจสอบพบว่าถูกต้องแล้วเครื่องแม่ข่ายจะเอาลายเซ็นนั้นๆ ออกและนำเอาลายเซ็นของตนเซ็นลงไปเมื่อจะเก็บข้อความนั้นลงใน log file

ทั้งคอนฟิगरชันไฟล์และฐานข้อมูลสามารถเก็บไว้ที่ส่วนกลางบนฝั่งของเครื่องแม่ข่ายได้และสามารถดาวน์โหลดไปใช้โดยเครื่องลูกข่ายในขณะที่ทำการ Startup เครื่องได้อีกด้วย

4.2.2 การไม่แสดงตัวตนว่ามีการใช้งานอยู่ (stealth)

Samhain มีความสามารถในการอำพรางตัวไม่ให้ผู้บุกรุกสามารถรู้ได้ว่ามีโปรเซสของ samhain กำลังทำงานอยู่ในระบบซึ่งการทำงานในลักษณะนี้เรียกว่าทำงานแบบ stealth ซึ่งสามารถทำได้โดยการ compile samhain ให้สนับสนุนโหมด stealth แต่การทำงานในโหมด stealth นั้นก็ยังอาจสามารถถูกตรวจสอบได้จากการค้นหาคอนฟิगरชันไฟล์ หรือค้นหาจากสตริงที่อยู่ในคั้งนั้น samhain จึงมี option ที่เพิ่มเติมเข้ามาคือ

- สตริงที่สามารถอ่านได้ซึ่งอยู่ใน executable file และที่อยู่ใน log และฐานข้อมูลสามารถทำให้คุณไม่รู้เรื่องได้ด้วยอย่างเช่น อาจทำให้มองดูแล้วเหมือนเป็นข้อมูลไบนารี
- สามารถยกเลิกการใช้ command line parsing ได้
- สามารถทำ Configuration data ให้เป็นรูปภาพโดยใช้ steganography
- Executable สามารถบีบอัดและเข้ารหัสลับได้

ฐานข้อมูลและ log file อาจถูกนำไปซ่อนโดยการนำไปต่อท้ายภาพที่มีอยู่แล้วก็ได้โดยภาพนั้นยังคงดูเหมือนปกติ

4.2.3 คุณสมบัติในการทำงานของ Samhain

- สามารถchecksumได้หลากหลายรูปแบบ เช่น TIGER192, SHA-1 ,MD5 เป็นต้น โดยจะตรวจสอบได้จากหลาย ส่วน เช่น size, mode/permission, owner, group, creation/modification/access time, inode, number of hardlinks, linked path(symbolic links) major/minor device number
- สามารถใช้ shell wildcard pattern ในการระบุไฟล์หรือไดเรกทอรีที่จะตรวจสอบได้
- สามารถตั้งและปรับแต่ง policies ของระบบได้ 8 รูปแบบที่แตกต่างกัน
- การตรวจสอบในเชิงลึกสามารถตั้งค่าให้ตรวจสอบทั้งหมด หรือตรวจสอบเฉพาะส่วนได้
- สามารถตรวจสอบความเปลี่ยนแปลงของระบบได้ว่ามีไฟล์ใดถูกแก้ไข หรือถูกลบไป

4.2.4 ตารางการตรวจเช็คของโปรแกรม

- ตรวจสอบเมื่อผู้ใช้สั่ง
- ตรวจสอบเมื่อมีการส่ง signal ไปยัง samhain daemon โดย signal อาจมาจากเครื่องที่ทำหน้าที่เป็น Samhain server
- สามารถสั่งให้ทำงานได้อัตโนมัติ เมื่อมีการใช้งานเครื่อง

4.2.5 ความสามารถในการตรวจสอบที่โดดเด่นอื่นๆ

Kernel integrity สามารถตรวจสอบ integrity ของ kernel ที่กำลังใช้งานอยู่ได้ เพื่อตรวจสอบ rootkits (สามารถใช้ความสามารถนี้ได้เฉพาะกับระบบปฏิบัติการ Linux หรือ FreeBSD แต่โดยไม่สามารถใช้กับ Fedora Core 2)

SUID/SGID files

- สามารถตรวจสอบการเปลี่ยนแปลงหรือสร้าง SUID/SGID files
- โดยสามารถสั่งให้ตรวจสอบอย่างสม่ำเสมอตามเวลาที่กำหนดได้
- สามารถลบหรือกักขังไฟล์ที่มีการกระทำดังกล่าวไว้ก่อนได้ โดยค่าที่เป็นค่าตั้งต้นจะกระทำเพียงแค่การเตือน

Mount check

- สามารถตรวจสอบการ mount ต่างๆ รวมทั้งการ mount files system.

Login/logoff events

- สามารถตรวจสอบการ login/logoff ของผู้ใช้งานได้โดยใช้ system file ที่ชื่อว่า utmp

Log facilities

- การกำหนดให้มีการจัดเก็บและแสดงการเตือน สามารถกำหนดได้
- เครื่องแม่ข่ายฐานข้อมูลจะได้รับข้อมูลผ่านทาง TCP connection ที่มีการเข้ารหัสลับไว้ และมีการ Authentication ระหว่างกัน เพื่อให้แน่ใจได้ว่าเป็นตัวจริงทั้งคู่

- สามารถใช้งาน Syslog
- สามารถใช้งานผ่าน Console ถ้าได้มีการติดตั้งตัว daemon ไว้
- Log files ที่จัดเก็บจะมีการ Sign ไว้เพื่อป้องกันการแก้ไขของผู้ไม่มีสิทธิ์
- E-mail ที่ใช้จะมีการ Sign ไว้เช่นเดียวกัน
- รองรับตัว RDBMS ที่หลากหลาย เช่น MySQL, PostgreSQL, และ Oracle

Integration with other monitoring systems

- **Nagios** : เป็น plugin เสริมที่สร้างขึ้นจาก Perl สำหรับตัว Nagios (check_samhain.pl)
- **Prelude** : สามารถ compiled เพื่อใช้งานในการตรวจสอบว่ามีแนวโน้มของระบบเป็นเช่นไร มีกลางบอกเหตุว่าจะเกิดเหตุการณ์ร้ายแรงหรือไม่

Integrity of the file checking system

เป็นเรื่องที่มักจะต้องเลือกเสมอระหว่างความปลอดภัยกับความสะดวกในการใช้งาน ที่ผ่านมาก็เคยมีการเก็บ โปรแกรมที่ใช้ตรวจสอบไว้บนเครื่องนั้นๆ แล้วหวังว่าจะไม่ถูกผู้บุกรุกเข้าไปแก้ไขหรือปลอมแปลง ซึ่งการทำเช่นนี้ก่อให้เกิดความไม่ปลอดภัยเกิดขึ้น ดังนั้น Samhain จึงใช้การ Sign ทั้งส่วนของ ฐานข้อมูลและส่วนของ configuration files เพื่อให้มีความถูกต้องของการทำงานโดยได้ทำการ Sign โดยใช้ GnuPG

4.3 Snort inline

Snort inline เป็นโปรแกรมที่ได้ถูกประยุกต์มาจากโปรแกรม Snort ซึ่งตัว Snort inline จะติดต่อกับแพ็คเก็ตจาก Iptables ผ่านทาง libipq แทนที่จะของเดิมที่จะรับจาก libpcap และกฎที่ใช้ในตัว Snort inline ก็จะเป็นรูปแบบของกฎใหม่ โดยมีการทำงาน drop, sdrops, reject โดยสั่งงาน Iptables ได้ 3 คำสั่งดังกล่าว ตัว Snort inline จึงเหมือนทำหน้าที่เป็น IPS (Intrusion Prevention System)

การกระทำอันเนื่องมาจากกฎที่ใช้

- **Drop** -The drop rule type
จะไปสั่งให้ iptables ทำการ drop แพ็คเก็ต และทำการเก็บ log ไว้
- **Reject** -The reject rule type
จะไปสั่งให้ iptables ทำการ drop แพ็คเก็ต และทำการเก็บ log ไว้ พร้อมทั้งส่ง TCP reset หรือ UDP reset กลับไป แล้วแต่กรณี

- **Sdrop -The sdrop rule type**

จะไปสั่งให้ iptables ทำการ drop แพ็คเก็ต โดยไม่ทำการบันทึกใดๆ

ลำดับในการพิจารณาของ Snort inline

->activation->dynamic->drop->sdrop->reject->alert->pass->log

สามารถใช้ทางเลือก -o เพื่อเปลี่ยนลำดับเป็น

->activation->dynamic->pass->drop->sdrop->reject->alert->log

Stream4 option การทำงานใหม่ใน Snort inline

ซึ่งสามารถเปิดการใช้งานในทางเลือก ได้อีก 2 ความสามารถดังนี้

- **inline_state (no arguments)**

เมื่อเปิดใช้งาน Snort inline จะทำการ drop แพ็คเก็ตที่ไม่เกี่ยวข้องหรือไม่สอดคล้องกับ TCP session ที่มีอยู่ และรวมถึงแพ็คเก็ตที่มี TCP initiator ผิดรูปแบบ

- **midstream_drop_alerts (no arguments)**

โดยค่ากำหนดเริ่มต้นสำหรับ Snort inline จะทำการ drop โดยไม่แสดงการเตือนใดๆ ถ้าหากเราต้องการให้มีการแจ้งเตือนก็ต้องเปิดโหมดการทำงานอันนี้ด้วย

ความสามารถในการแก้ไขข้อมูลในแพ็คเก็ต

นั่นคือเราสามารถสั่งให้มีการตรวจสอบ ข้อมูลที่ผ่านเข้าออกที่ผ่านตัว Snort inline ว่ามีค่าที่ตรงกับที่เราตั้งไว้หรือไม่ และถ้ามีก็ทำการเปลี่ยนให้เป็นไปตามที่เราต้องการ เช่น

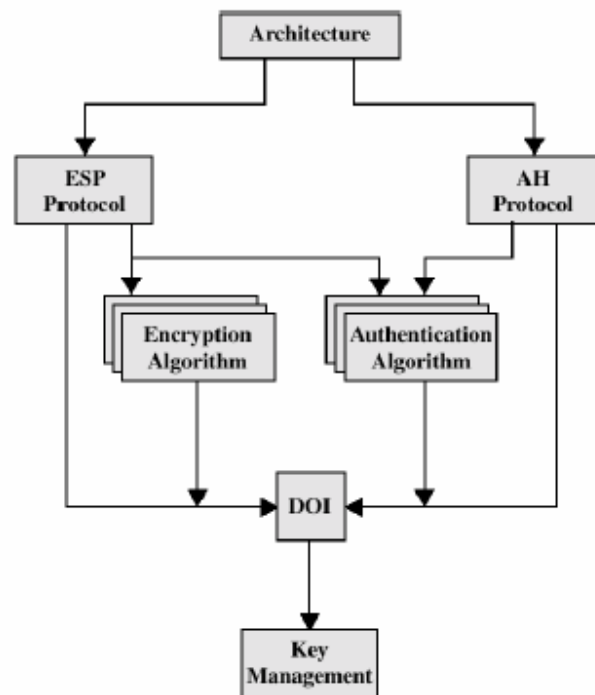
```
alert tcp any any <> any 80 (msg: "tcp replace"; content:"GET"; replace:"BET");
```

```
alert udp any any <> any 53 (msg: "udp replace"; content: "yahoo"; replace: "xxxxx");
```

ดังตัวอย่าง ตัว Snort inline ก็จะดูข้อมูลที่เป็น TCP ที่ใช้งาน port 80 โดยถ้าพบว่ามี ส่วนที่ตรงกับคำว่า GET ก็ให้เปลี่ยนเป็น BET และถ้าพบข้อมูลที่เป็น UDP ที่ใช้งาน port 53 ที่มีข้อมูลตรงกับคำว่า yahoo ก็ให้เปลี่ยนเป็น xxxxx

4.4 IPsec (IP security)

IPsec เป็นส่วนเพิ่มขยายของ Internet Protocol (IP) ในชุดโพรโทคอล TCP/IP
IPsec ใช้โพรโทคอล 2 ชุดคือ Authentication Header (AH) และ Encapsulated Security Payload (ESP) เพื่อรองรับการพิสูจน์ตัวตน(Authentication) การรักษาความถูกต้องของข้อมูล (Integrity) และ การรักษาความลับ (Confidentiality) ในระดับชั้นของ IP



รูปที่ 4.4 รูปภาพแสดงองค์ประกอบการทำงาน

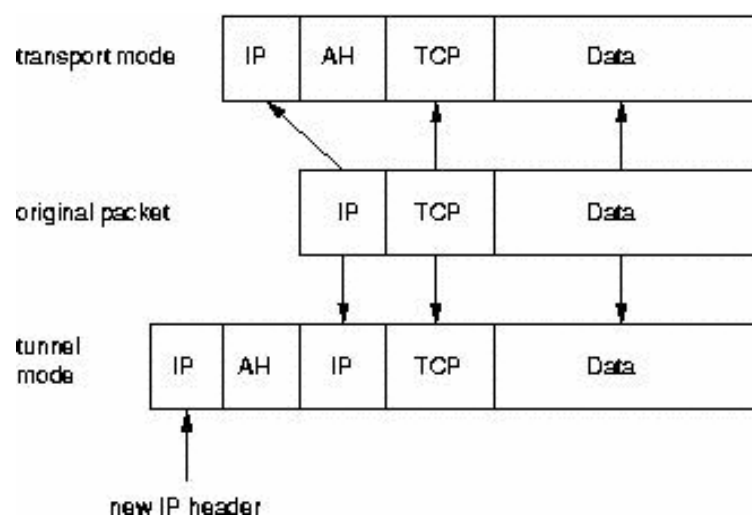
โดยการนำมาใช้งาน การทำส่วน AH จะแยกกับส่วน ESP โดยสามารถใช้ทั้งสองได้เพื่อประโยชน์ด้านความปลอดภัย โดยจะมีความสามารถดังตาราง

	AH	ESP (Encryption Only)	ESP (Encryption and Authentication)
Access Control	Y	Y	Y
Connectionless Integrity	Y		Y
Data Origin Authentication	Y		Y
Rejection of Replayed Packets	Y	Y	Y
Confidentiality		Y	Y
Limited Traffic Flow Confidentiality		Y	Y

ตารางที่ 4.1 ตารางแสดงความสามารถของ AH, ESP และ AH+ESP

โดยการใช้งานสามารถเลือกใช้ได้สองรูปแบบตามรูป

- **Tunnel mode** เป็นการนำส่วนแพ็กเก็ตเดิมทั้งหมดมาครอบด้วย IP โพรโตคอลชุดใหม่ที่ เป็นไปตามชุดโพรโตคอล IPsec สังเกตได้จากมีการเพิ่มเฮดเดอร์ IP และ AH เข้าไปข้างหน้า แพ็กเก็ตชุดเดิม
- **Transport mode** นำเฉพาะข้อมูลของโพรโตคอล IP ซึ่งจะประกอบด้วยข้อมูลของชั้น Transport (TCP หรือ UDP) และชั้นแอปพลิเคชัน โดยเพิ่มโพรโตคอล AH และเพิ่มข้อมูลใน IP เดิมให้เหมาะสมตามมาตรฐาน IPsec



รูปที่ 4.5 รูปแบบแพ็กเก็ตของ IPsec

การรักษาความถูกต้องของข้อมูลของ IP คาตาแกรม (IP Datagram) ในชุดโพรโทคอล IPsec ใช้ Hash Message Authentication Codes หรือ HMAC ด้วยฟังก์ชันแฮช เช่น MD5 หรือ SHA-1 ทุกครั้งที่มีการส่งแพ็กเก็ตจะมีการสร้าง HMAC และใช้การเข้ารหัสไปด้วยทุกครั้ง เพื่อให้ปลายทางสามารถตรวจสอบได้ตามหลักการลายเซ็นดิจิทัลว่าต้นทางเป็นผู้ส่งแพ็กเก็ตนั้นมาจริง ส่วนการรักษาความลับของข้อมูลนั้น จะใช้การเข้ารหัส IP คาตาแกรมด้วยวิธีการเข้ารหัสด้วยกุญแจสมมาตร ด้วยวิธีการมาตรฐานที่เป็นรู้จักกันดีเช่น 3DES AES หรือ Blowfish เป็นต้น

ปัญหาหนึ่งของ IPsec คือการส่งกุญแจที่ใช้ในการเข้ารหัสไปกับแพ็กเก็ต ซึ่งจัดว่าไม่ปลอดภัย นอกจากนี้การแลกเปลี่ยนกุญแจนำไปสู่ปัญหาของการดูแลระบบที่ใช้ IPsec เพราะทั้งระบบต้องสนับสนุนการใช้งานโพรโทคอล IPsec เดียวกัน จะทำอย่างไรให้สามารถส่งกุญแจในการเข้ารหัสไปกับแพ็กเก็ตถ้าไม่มีการเข้ารหัสแพ็กเก็ตแต่อย่างใด เพื่อแก้ปัญหานี้จึงได้พัฒนาโพรโทคอลในการแลกเปลี่ยนกุญแจหรือ Internet Key Exchange Protocol (IKE)

IKE จะทำการพิสูจน์ตัวตนของปลายทางก่อนการสื่อสาร ในขั้นตอนถัดมาจึงสามารถแลกเปลี่ยนและตกลง Security Association และกุญแจในการเข้ารหัสได้ด้วยวิธีการแลกเปลี่ยนกุญแจตามวิธีการแลกเปลี่ยนกุญแจด้วยการใช้กุญแจสาธารณะ เช่น Diffie-Hellmann เป็นต้น ซึ่งชุดโพรโทคอล IKE จะตรวจสอบกุญแจที่ใช้ในการเข้ารหัสระหว่างการติดต่อสื่อสารเป็นระยะตลอดการสื่อสารข้อมูลที่เกิดขึ้นแต่ละครั้ง

ชุดโพรโทคอล IPsec ประกอบจาก 2 โพรโทคอลหลักสองโพรโทคอลคือ

Authentication Header (AH) และ Encapsulated Security Payload (ESP)

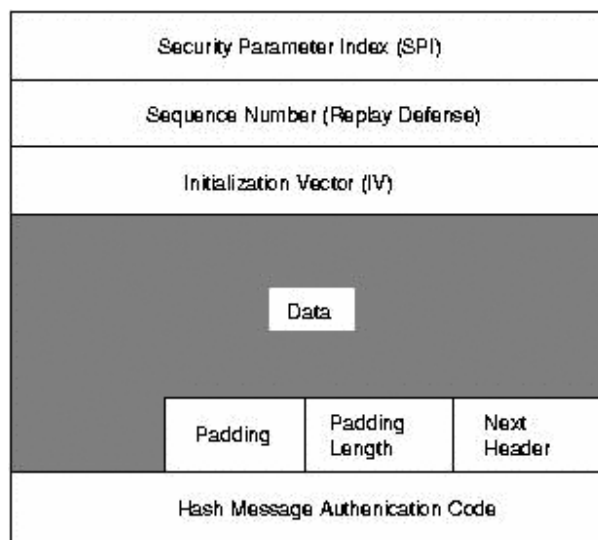
AH หรือ Authentication Header ทำหน้าที่รักษาความถูกต้องของ IP คาตาแกรม โดยการคำนวณ HMAC กับทุก IP คาตาแกรมตามรูป

Next Header	Payload Length	Reserved
Security Parameter Index (SPI)		
Sequence Number (Replay Defense)		
Hash Message Authentication Code		

รูปที่ 4.6 Authentication Header

เฮดเดอร์ของ AH มีขนาด 24 ไบต์ อธิบายได้ดังนี้

- Next Header ใช้เพื่อบอกให้ทราบว่ากำลังใช้รูปแบบใดในการใช้งาน IPsec ระหว่าง Tunnel mode ค่าจะเป็น 4 ส่วน Transport mode ค่าจะเป็น 6
- **Payload length** บอกความยาวของข้อมูลที่ต่อท้ายเฮดเดอร์ ตามด้วย Reserved จำนวน 2 ไบต์
- **Security Parameter Index (SPI)** กำหนด Security Association สำหรับใช้ในการถอดรหัสแพ็กเก็ตเมื่อถึงปลายทาง
- **Sequence Number** ขนาด 32 บิตใช้บอกลำดับของแพ็กเก็ต
- **Hash Message Authentication Code (HMAC)** เป็นค่าที่เกิดจากฟังก์ชันแฮชเช่น MD5 หรือ SHA-1 เป็นต้น
- **ESP หรือ Encapsulated Security Payload** ใช้สำหรับรักษาความถูกต้องของแพ็กเก็ต โดยใช้ HMAC และการเข้ารหัสรวมด้วย



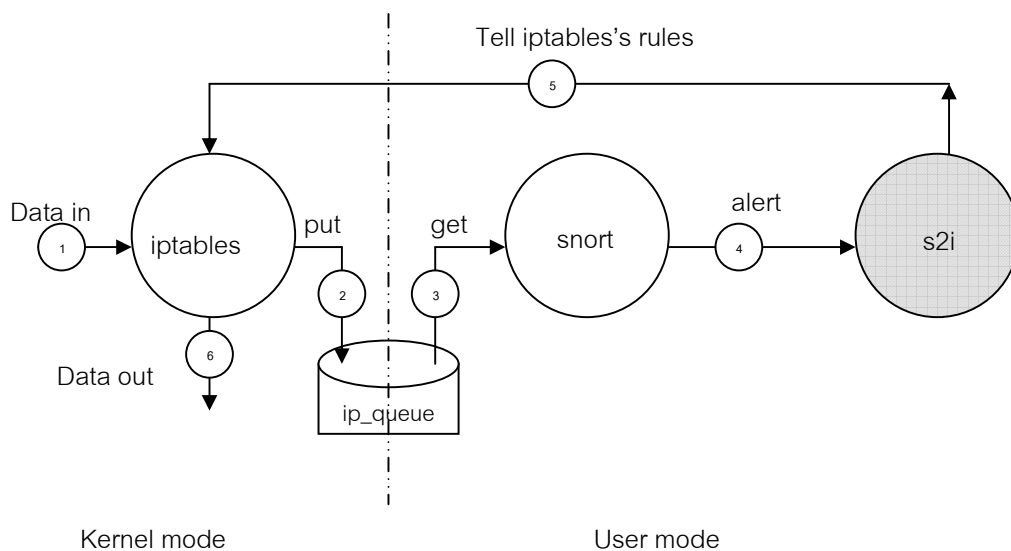
รูปที่ 4.7 Encapsulated Security Payload

- **Security Parameter Index (SPI)** ใช้ในการกำหนด Security Association (SA) ระบุ ESP ที่สอดคล้องกัน
- **Sequence Number** ระบุลำดับของแพ็กเก็ต

- **Initialization Vector (IV)** ใช้ในกระบวนการเข้ารหัสข้อมูล ป้องกันไม่ให้สองแพ็กเก็ตมีการเข้ารหัสที่ซ้ำกันเกิดขึ้น
- **Data** คือข้อมูลที่เข้ารหัส
- **Padding** เป็นการเติม Data เพื่อให้ครบจำนวน ไบต์ที่เข้ารหัสได้
- **Padding Length** บอกความยาวของ Padding ที่เพิ่ม
- **Next Header** กำหนดเฮดเดอร์ถัดไป
- **HMAC** ค่าที่เกิดจากฟังก์ชันแฮชขนาด 96 บิต

4.5 S2I (S2I: Snort command to iptables)

เป็นโปรแกรมที่สร้างจากภาษาคอมไพเลอร์ เพื่อให้สามารถแปลงการแจ้งเตือนของตัวระบบตรวจจับผู้บุกรุก ให้แปลงเป็นคำสั่งของไฟร์วอลล์ (ไอพีเทเบิล) ได้อย่างเหมาะสม เพื่อให้ชั้นข้อมูลที่มาจากผู้ใช้งานปกติไปยังส่วนที่ต้องการตามต้องการ และในส่วนของผู้บุกรุกเองนั้นก็ส่งไปยังเครื่องกับดัก ตัวเอสทูไอสามารถกำหนดเปลี่ยนแปลงค่าต่างๆ ได้เพื่อให้เกิดความเหมาะสมในการใช้งานกับระบบเครือข่ายนั้นๆ ซึ่งมีโปรแกรมเสริมดังนี้



รูปที่ 4.8 โค้ดอะแกรมแสดงการทำงานของ S2I

4.6 Iptables

ไอพีเทเบิลนั้นเป็นโปรแกรมไฟร์วอลล์แบบ State Full Inspector ที่ทำงานใน Kernel-Mode โดย iptables จะมีหน้าที่ในการจัดการเชื่อมต่อ และการอนุญาตหรือไม่อนุญาตให้ชั้นข้อมูลต่างๆ สามารถผ่านเข้าออกระบบเครือข่ายได้ ซึ่งในโครงการนี้ได้นำโปรแกรม iptables มาใช้งานเพื่อให้ชั้นข้อมูลที่ผ่านเข้ามาในระบบเครือข่ายถูกส่งไปให้ส่วนที่โปรแกรมตรวจจับผู้บุกรุกที่ทำงานในยูสเซอร์ โหมดนำชั้นข้อมูลเหล่านั้นไปพิจารณาต่อไป โดยชั้นข้อมูลจะถูกส่งผ่านทางโมดูลที่ชื่อว่า ip_queue และอีกหน้าที่หนึ่งของ iptables คือการจัดเส้นทางการเชื่อมต่อให้กับผู้ใช้ที่ถูกจำแนกแล้วว่า เป็นผู้ใช้ปกติหรือเป็นผู้บุกรุกระบบ ให้เข้าไปใช้งานเครื่องให้บริการจริง หรือเครื่องกักดัก โดยตัวไอพีเทเบิลจำเป็นจะต้องทำงานร่วมกับ Snort_inline โดยผ่านการเชื่อมให้สามารถทำงานประสานกันได้ โดย S2I

Linux สามารถใช้งานเป็นไฟร์วอลล์ได้ตั้งแต่เคอร์เนล 1.1 ซึ่งเป็นเวอร์ชันแรก โดย Alan Cox ใช้ชื่อว่า ipfw (จาก BSD) ต่อมา Linux 2.0 ได้ถูกพัฒนาและปรับปรุงได้เครื่องมือที่มีชื่อว่า ipfwadm โดยเครื่องมือชิ้นนี้อินทิเกรตให้ผู้ใช้สามารถควบคุม filtering rule ได้ และต่อมา Linux 2.2 ก็ได้สร้างเครื่องมือตัวใหม่ชื่อ ipchains ซึ่งเผยแพร่ในปี 1998 โดย Rusty Russel และทีมงาน ทั้งนี้ ipchains นี้ถือได้ว่าเป็นพัฒนาการขั้นที่สามของ Linux Firewall จวบจนกระทั่งในปัจจุบัน ก็มี netfilter และ iptables ซึ่งถือว่าเป็นพัฒนาการขั้นที่สี่ของ Linux Firewall

Netfilter นั้นเป็นชื่อใหม่ของโค้ดที่ทำหน้าที่เป็น packet handler(stateful inspection) ใน Linux kernel 2.4 (จริงคือเวอร์ชัน 2.3.15 และเวอร์ชันต่อๆ มา) ซึ่งได้ถูกออกแบบและปรับปรุงใหม่จากเวอร์ชันก่อนหน้านี้ เป็นเรื่องที่น่ายินดีคือ netfilter นั้นสามารถทำงานย้อนหลังร่วมกับ ipchains และ ipfwadm ได้ และคำสั่งในการเรียกใช้งานคือ iptables

ความแตกต่างระหว่าง iptables และ ipchains

- ชื่อของ built-in chain (ประกอบไปด้วย INPUT, OUTPUT, FORWARD) เปลี่ยนจากตัวอักษรเล็ก (lowercase) เป็นตัวอักษรใหญ่ (uppercase)
- การใช้งานที่ต้องระบุ port ทั้ง TCP และ UDP นั้น ต้องใช้คำว่า --source-port หรือ --sport (--destination-port หรือ --dport) และต้องใช้ตามหลังจาก -p tcp หรือ -p udp
- TCP -y flag เปลี่ยนเป็น --syn และต้องใช้ร่วมกับ -p tcp
- target จาก DENY เปลี่ยนเป็น DROP

- chain ที่ไม่มี rule ใดๆ เลขก็สามารถทำงานได้
- การทำ zeroing built-in chain จะทำให้ byte counter ถูกล้างค่าไปด้วย
- ชื่อของ chain ยาวสูงสุดได้ 31 ตัวอักษร
- MASQ เปลี่ยนเป็น MASQUERADE และมีรูปแบบการใช้งานเปลี่ยนไป รวมทั้ง REDIRECT ก็มีการเปลี่ยนแปลงรูปแบบใหม่

รูปแบบการใช้งาน iptables เบื้องต้น

iptables จะมีรูปแบบการใช้งานดังนี้คือ

iptables [table] <command> <match> <target/jump>

โดย rule ที่เขียนขึ้นจะเป็นเป็นตัวบอกเครื่องเฝ้าให้กระทำ action อย่างไร ในกรณีที่พบ packet ตรงตามที่ระบุไว้

- **[table]** หมายถึง ตารางหรือ table ที่ต้องการระบุ เช่น iptables -t nat หมายถึงให้ทำงานกับ nat table ในกรณีที่ไม่ได้ระบุตาราง iptables จะถือว่าคำสั่งดังกล่าวระบุถึง filter table โดยอัตโนมัติ
- **<command>** จะเป็นตัวสั่งให้ iptables ทำในสิ่งที่ต้องการ เช่น iptables -A INPUT ซึ่งหมายถึงให้สร้าง rule ต่อท้าย INPUT chain ใน filter table
- **<match>** เป็นส่วนที่ใช้ตรวจสอบว่า packet มีข้อมูลตรง (match) กับที่ระบุไว้หรือไม่ เช่น มี source ip address เป็น 1.2.3.4
- **<target/jump>** เป็นตัวระบุว่าเมื่อเจอ packet ที่ match ก็จะทำ (action) ตามที่ระบุไว้ เช่น ถ้า packet ใดมี source ip address เป็น 1.2.3.4 ให้ DROP packet นั้นทิ้งไป

Table

iptables สามารถทำงานได้กับตาราง(table) 3 ตารางหลัก สามารถระบุตารางได้โดยใช้ 옵션 -t ตามด้วยชื่อ table คือ

1. **Filter table** ใช้สำหรับกรอง packet มี 3 built-in chain คือ INPUT, OUTPUT, FORWARD ซึ่งจะได้อธิบายรายละเอียดต่อไป
2. **Nat table** ใช้สำหรับการแปลงแอดเดรส (Network Address Translation) มี 3 built-in chain คือ PREROUTING, POSTROUTING, OUTPUT ซึ่งรายละเอียดจะได้อธิบายต่อไป

3. **Mangle table** เป็นตารางที่ใช้เปลี่ยนแปลงหรือแก้ไข packet เช่น เปลี่ยนค่า TTL, MARK ซึ่งปกติจะใช้ในการทำ routing ที่มีความซับซ้อนสูง มี 2 built-in chain คือ PREROUTING chain (ใช้แก้ไข packet ก่อนที่จะเข้าสู่ไฟร์วอลล์และก่อนเข้าสู่ routing decision) และ OUTPUT chain (ใช้แก้ไข packet ที่ถูกสร้างโดยไฟร์วอลล์ก่อนที่มันจะถูกส่งไปยัง routing decision) ทั้งนี้ไม่สามารถทำ network address translation หรือ masquerading ที่ table นี้ได้ และในเอกสารฉบับนี้จะไม่กล่าวถึง mangle อีก เนื่องจากเป็นส่วนที่ไม่นิยมนำไปใช้งาน

Match

การตั้งเงื่อนไขของการ match นั้นจะต้องอาศัยความเข้าใจในเรื่อง IP, TCP, UDP, และ ICMP มาบ้างพอสมควร จึงจะสามารถตั้งเงื่อนไขที่เหมาะสมและตรงตามความต้องการได้ ซึ่งมีรายละเอียดดังนี้

● การระบุ source, destination IP address

สามารถระบุ source ip address ของ packet โดยใช้ **-s** หรือ **--source** หรือ **--src** และสำหรับ destination ip address ก็ใช้ **-d** หรือ **--destination** หรือ **--dst** การระบุไอพีแอดเดรสนั้นสามารถทำได้ 4 แบบด้วยกันคือ

1. ใช้ชื่อเต็มแทน เช่น localhost หรือ www.nectec.or.th
2. ระบุไอพีแอดเดรสโดยตรง เช่น 127.0.0.1 หรือ 202.44.204.33
3. ระบุเป็น group ของไอพีแอดเดรส เช่น 202.44.204.0/24 ซึ่งหมายถึงไอพีแอดเดรสตั้งแต่ 202.44.204.0 - 202.44.204.255
4. หรืออาจจะใช้ 202.44.204.0/255.255.255.0 แทน 202.44.204.0/24 ได้

● การทำ Inversion

ในบางกรณีนั้นหากต้องการระบุเป็น inverse เช่น อนุญาตให้ทุกไอพียกเว้นไอพีที่ระบุไว้ ซึ่งการใช้คำสั่งดังกล่าวสามารถทำได้โดยใช้เครื่องหมาย **!** นำหน้า argument ที่ต้องการ (เครื่องหมาย **!** หมายถึง NOT) เช่น **-p ! TCP** ซึ่งจะ match กับโปรโตคอลทุกๆ ตัวที่ไม่ใช่ TCP หรือ **-s ! localhost** ซึ่งหมายถึง packet ที่มี source ip address อื่นๆ ยกเว้น localhost (127.0.0.1)

● การระบุโปรโตคอล

สามารถระบุโปรโตคอลที่ต้องการได้ดังนี้คือ TCP, UDP, ICMP หรือสามารถใช้ตัวเลขแทนได้ (สำหรับ *NIX อ้างอิงได้จาก /etc/protocols) และยังสามารถใช้ได้ทั้งตัวอักษรเล็กหรือใหญ่ (ใช้ได้ทั้ง tcp และ TCP) เช่น **-p TCP** หรือ **-p ! tcp**

- **การระบุ interface**
-i หรือ **--in-interface** ตามด้วยชื่อ interface ใช้เพื่อระบุ incoming interface ซึ่งหมายถึงว่า packet ที่จะ match กับ rule นี้ต้องเข้ามาจาก interface ที่กำหนด เช่น -i eth0 หมายความว่า ทุก packet ที่เข้ามาทาง eth0 จะ match กับ rule นี้ ทั้งนี้ชื่อ interface ที่สามารถใช้ได้นั้น สามารถตรวจสอบได้โดยใช้คำสั่ง ifconfig และ **-o** หรือ **--out-interface** ตามด้วยชื่อของ interface ใช้เพื่อระบุ outgoing interface ซึ่งหมายถึงว่า packet ที่จะ match กับ rule นี้ กำลังจะเดินทางผ่าน interface ที่ระบุไว้ เช่น -o eth1 หรือ -o ! eth1

ข้อสังเกต

- สำหรับ INPUT chain นั้นไม่มี output interface ดังนั้นหากใช้ -o ร่วมกับ INPUT chain ก็จะไม่ packet ใดที่ match กับ rule นี้เลย
- ทำนองเดียวกันกับ OUTPUT chain ที่ไม่มี input interface ดังนั้นหากใช้ -i ร่วมกับ OUTPUT chain ก็ไม่มีประโยชน์อันใด
- FORWARD chain มีได้ทั้ง input และ output interface
- หากระบุ interface ที่ไม่มีอยู่จริง ก็จะไม่ packet ใดที่ match กับ rule นั้นเลย
- หากใช้เครื่องหมาย + ร่วมกับ interface เช่น ppp+ นั้นจะหมายถึงทุกๆ ppp interface เช่น ppp0, ppp1

- **fragment packet**
 ในการส่งข้อมูลใน ip network นั้นเป็นเรื่องปกติที่จะเกิดการ fragment ของ packet เนื่องจากขนาดของ packet มีขนาดใหญ่เกินไปที่จะส่งไปในครั้งเดียว จำเป็นต้องมีการแบ่ง packet ออกเป็นหลายๆ ชิ้นทยอยส่งไป ซึ่งเรียกกันว่าการทำ fragment โดยเครื่องปลายทางจะทำหน้าที่ประกอบ fragment packet รวมกันเป็น packet ที่สมบูรณ์ดั้งเดิม ข้อมูลที่เป็น fragment packet นั้นจะมี header ที่สมบูรณ์แค่ packet แรกเท่านั้น ตัว packet ที่ตามมาจะมีแค่ header บางส่วนคือ ไอพีแอดเดรสเท่านั้น ไม่มีข้อมูลของโปรโตคอลแนบมาด้วย ดังนั้นการตรวจสอบข้อมูล header ของ TCP, UDP, ICMP จึงไม่สามารถทำได้ใน packet ที่สองเป็นต้นมา

หากใช้ NAT บรรดา fragment packet จะถูกประกอบเข้าด้วยกันจนสมบูรณ์ก่อนที่ packet จะเข้าไปถึง packet filtering ดังนั้นจึงไม่มีความจำเป็นที่จะต้องกังวลเกี่ยวกับ fragment packet

ดังนั้นถ้าไม่ได้ใช้ NAT ก็ควรทำความเข้าใจไว้ว่า iptables มีกระบวนการในการทำงานกับ fragment packet อย่างไร หลังจากที่ fragment packet แรกผ่านเข้ามาแล้ว iptables สามารถตรวจสอบได้ว่าจะอนุญาตให้ผ่านหรือไม่ ในขณะที่ fragment packet ที่สองและหลังจากนั้นที่ตามมานั้น จะไม่สามารถ match กับ rule ใดๆ เลย เช่น -p TCP --sport www หรือแม้แต่ -p TCP --sport ! www

อย่างไรก็ตาม สามารถเขียน rule ให้ตรวจสอบทั้ง fragment packet ตัวที่สองและหลังจากนั้นที่ตามมาได้ด้วยการใช้ -f หรือ --fragment ทั้งนี้อาจจะเขียนในทางตรงข้ามก็ไม่ต้องตรวจสอบ fragment packet ที่สองและหลังจากนั้นโดยใช้ ! -f ก็ได้ ทั้งนี้โดยปกติแล้วมักจะปล่อยให้ fragment packet ผ่านไป เนื่องจากถ้าสามารถ DROP ตัว fragment packet ตัวแรกได้แล้ว มันก็ไม่สามารถถูกประกอบที่เครื่องปลายทางได้ แต่ทั้งนี้ fragment packet ที่ถูกปล่อยไปดังกล่าวอาจจะทำให้เครื่องที่ได้รับ hang หรือ crash ได้ หรืออาจจะเกิดการโจมตีแบบ Denial of Service โดยใช้ fragment packet ได้

คำแนะนำเพิ่มเติม

- ควรมีการป้องกันการปลอมไอพี(IP spoof) สำหรับเครื่องไฟร์วอลล์ ซึ่งสามารถรัน script ด้านล่างนี้เพื่อป้องกันปัญหาดังกล่าวได้

```
for x in lo eth0 eth1
do
    echo 1 > /proc/sys/net/ipv4/conf/${x}/rp_filter
done
```

แต่ script ดังกล่าวนี้อาจไม่ได้ป้องกันการปลอมไอพีสำหรับทั้งเครือข่าย ดังนั้นจึงต้องสร้าง rule ไว้ที่ PREROUTING chain เพื่อป้องกันปัญหานี้อีกครั้ง เช่น

```
SIPTABLES -t nat -A PREROUTING -i $INET_IFACE -s 10.0.0.0/8 -j DROP
SIPTABLES -t nat -A PREROUTING -i $INET_IFACE -s 127.0.0.0/8 -j
DROP
SIPTABLES -t nat -A PREROUTING -i $INET_IFACE -s 172.16.0.0/12 -j
DROP
```

\$IPTABLES -t nat -A PREROUTING -i \$INET_IFACE -s 192.168.1.0/16 -j DROP

การป้องกันการปลอมไอพีดัง rule ด้านบนนี้ สามารถลดปัญหาที่จะเกิดขึ้นจากการโจมตีแบบ Denial of Service เช่น Land attack ได้เป็นอย่างดี

- การปิดการทำงานของโปรโตคอล ICMP เพราะถือได้ว่าเป็นโปรโตคอลที่ไม่มีความปลอดภัย การโจมตีที่เกิดขึ้นแบบ Denial of Service ในช่วงที่ผ่านมาก็ใช้ ICMP เป็นหลัก เช่น Smurf attack, Ping flood(Ping of Death) แต่การยกเลิกการใช้งาน ICMP อาจจะทำให้เกิดความไม่สะดวกในการใช้งาน ซึ่งก็ขึ้นอยู่กับวิจารณญาณของผู้ดูแลระบบเครือข่ายด้วย
- เนื่องจากการสร้าง policy ของแต่ละ chain นั้นสามารถเลือกได้ 2 แบบ คือ DROP และ ACCEPT ซึ่งรูปแบบการเขียน rule สำหรับแต่ละ policy นั้นก็แตกต่างกันเล็กน้อย แต่ระดับของความปลอดภัยที่ได้ถือได้ว่าเป็นความแตกต่างกันพอสมควรคือ
 - ถ้าหากใช้ policy เป็น DROP แล้ว ก็เสมือนเป็นระบบปิด การสร้าง rule ก็เพียงแค่ ACCEPT สำหรับ port หรือ service ที่ต้องการให้เปิดใช้เท่านั้น
 - ในขณะที่หากใช้ policy เป็น ACCEPT แล้ว ก็เสมือนเป็นระบบเปิด จะต้องสร้าง rule สำหรับปิด service หรือ port บางส่วน ซึ่งมีโอกาสที่จะเกิดความผิดพลาดได้ง่ายกว่า
- ควรมีการทำ hardening OS สำหรับเครื่องที่จะนำมาใช้เป็นไฟร์วอลล์ รวมทั้งจำกัดการเข้าถึงเครื่องนี้จากภายนอก ติดตั้ง service เฉพาะที่จำเป็นเท่านั้น และติดตั้งเครื่องมือช่วยสำหรับผู้ดูแลระบบเพื่อตรวจตราความผิดปกติที่อาจเกิดขึ้นในระดับ host based เพราะถ้าผู้บุกรุกสามารถเจาะเข้ามายังเครื่องไฟร์วอลล์ได้แล้วก็เสมือนกับสามารถควบคุมเครือข่ายได้ทั้งหมด
- ติดตั้ง NIDS เช่น Snort ในเครือข่ายทั้งหน้าและหลังไฟร์วอลล์ ซึ่งเครื่องที่จะติดตั้ง NIDS ก็ควรจะมีประสิทธิภาพเช่นเดียวกับเครื่องที่ติดตั้งไฟร์วอลล์
- ภายหลังที่สร้าง rule เสร็จแล้ว ควรใช้ port scanner ทำการ scan เครื่องทั้งหมดจากภายนอก โดยให้ scan ทุกโปรโตคอลคือ TCP, UDP และ ICMP scan เพื่อเป็นการตรวจสอบเบื้องต้นว่าไฟร์วอลล์ทำงานตามที่ได้ตั้งไว้หรือไม่

- การสร้าง rule บางอย่างนั้น จะต้องมีความรู้ในเรื่องของโปรโตคอลทั้ง TCP, UDP, ICMP เป็นอย่างดี เพราะไม่เช่นนั้นแล้วก็จะเป็นการเปิดช่องโหว่ในระบบโดยไม่รู้ตัว และ rule ในไฟร์วอลล์ที่ดีนั้นจะต้องไม่มีจำนวนมากเกินไป เพื่อป้องกันไม่ให้เครื่องที่รันไฟร์วอลล์ทำงานหนักเกินไป รวมทั้งป้องกันไม่ให้เกิด human error เนื่องจากความยาวที่มากเกินไป
- ขอให้ผู้ที่นำไฟร์วอลล์ไปใช้งานระลึกไว้เสมอว่า ไฟร์วอลล์ไม่สามารถป้องกันการบุกรุกระบบได้อย่างสมบูรณ์ 100% เพราะไฟร์วอลล์เป็นเพียงอุปกรณ์ที่ใช้เพื่อลดความเสี่ยงเท่านั้น